APPROACH 2

MODULAR VHDL

8-bit Accumulator CPU

A step-by-step architecture, datapath, control, timing, truth tables, implementation workflow and viva guide

University of Asia Pacific
Department of Computer Science and Engineering
CSE 450 — VLSI Lab • Final Project • Spring 2026

Student ID: 22201054 **Assigned Set:** 4

Mandatory: LDI, ADD **Assigned:** STA, LDR, AND

Design principle: use VHDL entities and behavioral descriptions for the required CPU modules. No NAND-only construction is used in this approach.

Prepared from the uploaded assignment brief and the supplied Approach 1 guide, rewritten for the modular VHDL implementation path.

Clickable guide contents

Use the PDF outline/bookmarks or the section list below to navigate. The guide keeps the same step-by-step report style as Approach 1, but starts directly from the required modular architecture.

- 1. Assignment requirements and Set 4 scope
- 2. CPU specification and instruction format
- 3. Modular architecture and hierarchy
- 4. Instruction execution pipeline / dataflow
- 5. PC and Program ROM
- 6. Control Unit and control truth table
- 7. ALU and datapath
- 8. ACC, R and Z registers
- 9. Clock, reset and timing diagram
- 10. Truth tables and worked examples
- 11. Exact Set 4 ROM contents
- 12. Cycle-by-cycle expected results
- 13. VHDL project structure and implementation order
- 14. VHDL module templates / integration patterns
- 15. Testbench plan and waveform checklist
- 16. Debugging checklist
- 17. Viva questions and answers
- 18. Final presentation pipeline and conclusion

Scope note

The assignment defines a single-cycle CPU: fetch, decode and execute occur within one instruction cycle, with register state committed at the rising edge. The word “pipeline” in this report refers to the logical sequence/dataflow of that single-cycle instruction—not a multi-stage pipelined processor with inter-stage registers.

1. Assignment requirements and Set 4 scope

The project brief asks for a small single-cycle 8-bit accumulator CPU implemented in VHDL. It explicitly asks for separate modules for the Program Counter (PC), Program ROM, Control Unit, ALU and ACC/R/Z registers, all connected by a top-level CPU entity. A testbench must verify the supplied program.

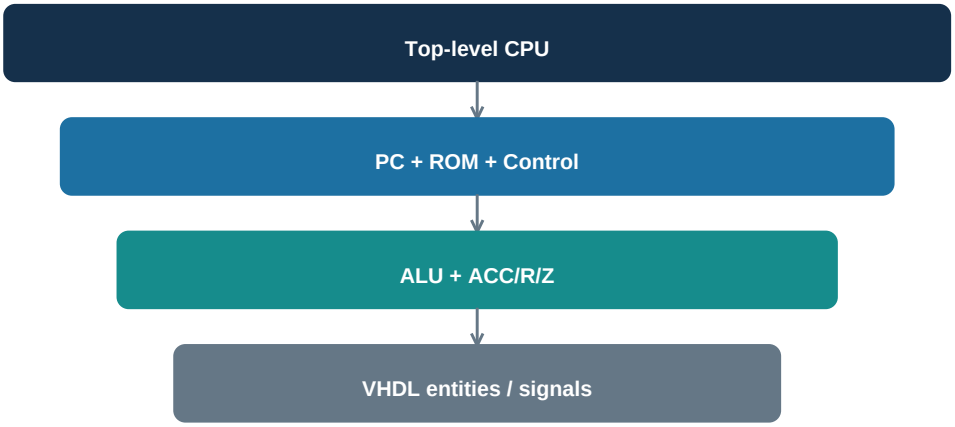
Requirement	Specification
CPU style	Single-cycle 8-bit accumulator CPU
Clocking	PC, ACC, R and Z update on rising_edge(CLK)
Reset	Synchronous, active-high; reset has priority
PC	4 bits; addresses 16 ROM entries
ROM	16 locations × 12 bits; unused locations are 0x000
Instruction	12 bits = opcode[11:8] + operand[7:0]
Outputs	CLK, RESET, Q[7:0] (ACC), Z and PC[3:0]
Set 4 instructions	LDI, ADD, STA, LDR, AND

Set 4 is determined by the final digit 4 in student ID 22201054. The two mandatory instructions are LDI and ADD; the assigned instructions are STA, LDR and AND.

Do not add SUB, OR, XOR, NOT, shifts or JZ to this implementation. The brief says to implement only the five instructions in the assigned set. Any unsupported opcode, including 0000, behaves as NOP: architectural data state is unchanged and PC advances by one.

2. CPU specification and instruction format

The instruction word is 12 bits wide. The most-significant four bits select the operation; the low eight bits carry an immediate value or are ignored for register-only instructions.



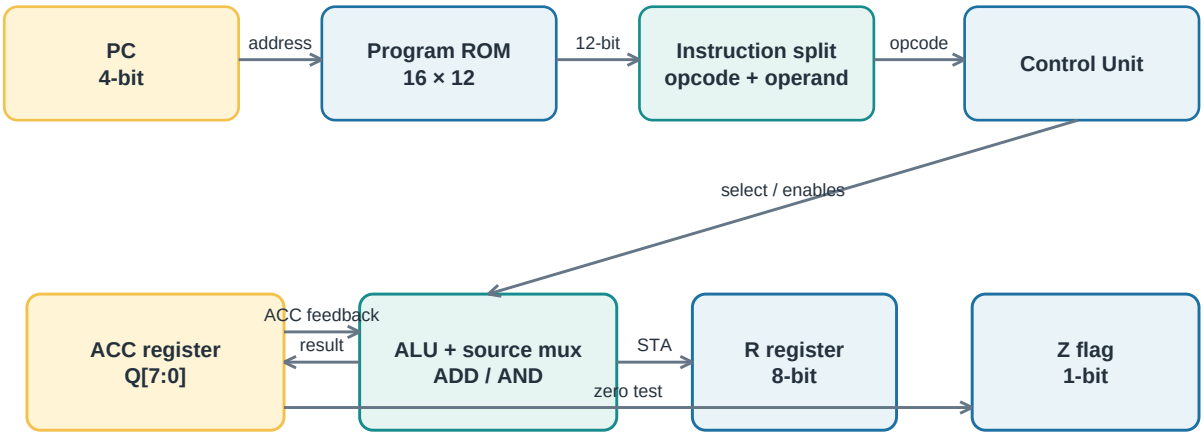
Field	Width	Meaning
instr[11:8]	4 bits	Opcode / instruction class
instr[7:0]	8 bits	Immediate operand or unused field
PC[3:0]	4 bits	Address of instruction in ROM
ACC / Q[7:0]	8 bits	Accumulator and externally visible CPU result
R[7:0]	8 bits	Holding register used by STA and LDR
Z	1 bit	1 if the most recently written ACC value is zero; unchanged by STA

Example encoding: LDI 0x1E uses opcode 0001 and operand 00011110. The 12-bit word is 0001 00011110 = 0x11E.

Arithmetic is 8-bit. ADD stores only the low eight bits; any ninth carry bit is discarded. Z is updated for LDI, ADD, AND and LDR based on the new ACC value. STA does not change ACC or Z.

3. Modular architecture and hierarchy

Approach 2 describes each required block as a VHDL entity. The top-level CPU wires the entity ports together. VHDL handles arithmetic, Boolean operators, register processes and selection logic; the design is not manually assembled from NAND gates.



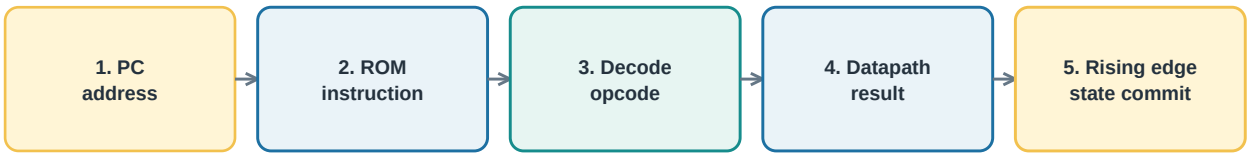
All state registers update only at rising_edge(CLK); RESET is synchronous and active-high.

Entity / block	Inputs	Responsibility
pc	CLK, RESET, next-PC	Holds 4-bit address; increments normally
program_rom	address[3:0]	Combinationally returns 12-bit instruction
control_unit	opcode, Z	Generates enables, ALU select and PC control
alu	ACC, operand, operation	Computes ADD or AND result
registers	CLK, RESET, enables, data	Stores ACC, R and Z
cpu_top	CLK, RESET	Connects modules and exposes Q, Z, PC
tb_cpu	simulation stimuli	Drives clock/reset and verifies state trace

Design boundary: combinational logic computes signals between clock edges; sequential logic changes the state only on the rising edge. ROM is combinational because the assignment says no clock is needed for it.

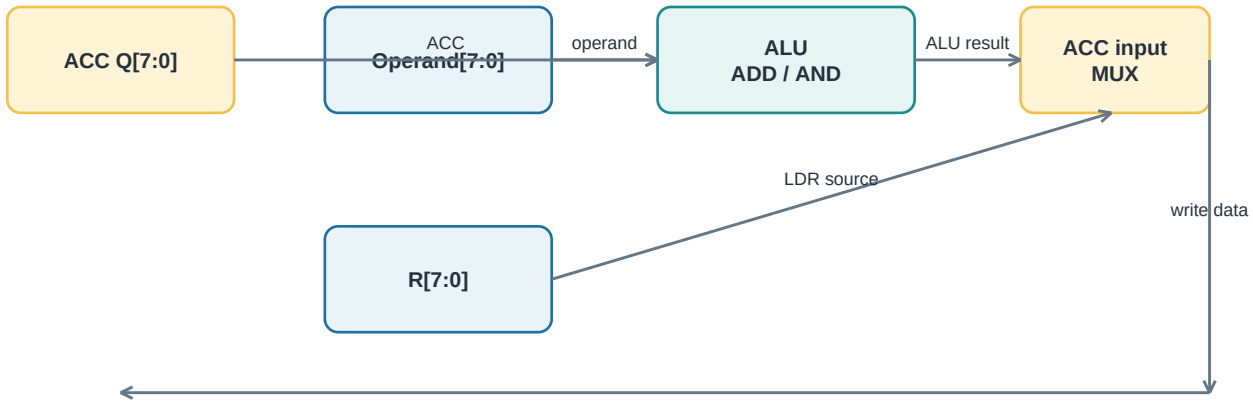
4. Instruction execution pipeline and dataflow

The instruction path is easiest to explain as five logical stages. These stages are not separate clock cycles: all combinational stages settle during one instruction cycle, then the selected registers capture their next values at the active edge.



Single-cycle design: stages are conceptual parts of one instruction cycle, not separate pipeline registers.

1. Fetch: PC selects ROM[PC]. 2. Decode: split instr into opcode and operand. 3. Control: generate ACC write, R write, Z update, ALU operation and next-PC selection. 4. Execute/data select: ALU or immediate/R path produces ACC input; STA routes ACC to R. 5. Commit: at rising_edge(CLK), enabled state registers capture values.



LDI selects immediate; LDR selects R; ADD/AND select the ALU result.

Dataflow by instruction: LDI routes the immediate operand to ACC; ADD routes ACC and operand through the adder; AND routes ACC and operand through bitwise AND; STA copies ACC into R while ACC holds; LDR routes R into ACC. The PC advances to PC+1 for every Set 4 instruction.

Important terminology

A true pipelined CPU usually has stage registers and can overlap multiple instructions. This assignment explicitly describes a single-cycle CPU. Use “instruction dataflow” or “logical execution pipeline” in your presentation to avoid implying a multi-cycle pipeline.

5. Program Counter and Program ROM

Program Counter (PC): a 4-bit register representing the address of the next instruction to execute. Since Set 4 contains no JZ, normal execution is PC ← PC + 1. Four-bit arithmetic wraps from 15 to 0.

Reset behavior: if RESET=1 at a rising edge, PC becomes 0 regardless of the instruction. If RESET=0, PC increments by one at each rising edge.

Program ROM: a constant 16-element array of 12-bit words. It is read combinationally using the current PC. Locations 0–9 contain the required Set 4 program; locations 10–15 contain 0x000 (NOP).

PC condition at rising edge	Next PC	Reason
RESET = 1	0000	Synchronous reset takes priority
RESET = 0	PC + 1 (4-bit)	No JZ in Set 4
PC = 1111 and RESET = 0	0000	4-bit wrap-around

ROM implementation notes

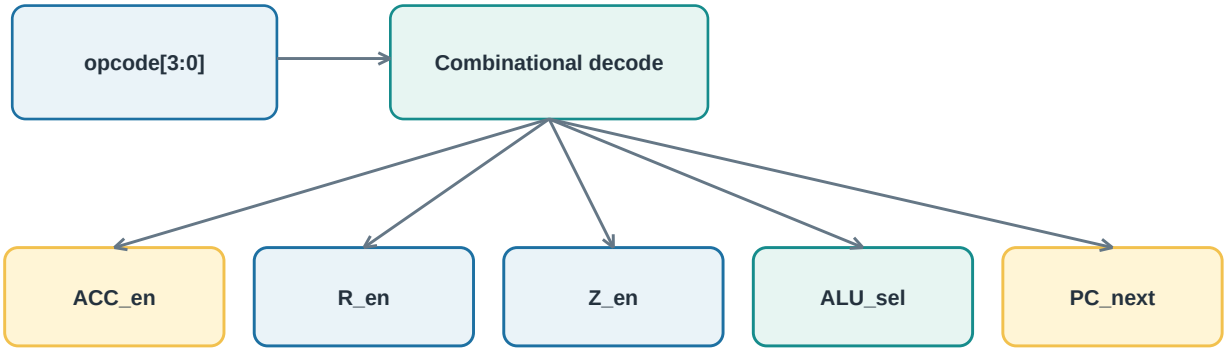
Use a constant array or a combinational case statement. Avoid adding a clock to the ROM. Use the exact encoded words in Section 11. Default unspecified addresses to x"000". A ROM word is not an integer instruction index; it is the full 12-bit opcode-plus-operand encoding.

VHDL idiom

```
type rom_t is array (0 to 15) of std_logic_vector(11 downto 0);
constant ROM : rom_t := (
  0 => x"11E", 1 => x"A00", 2 => x"222", 3 => x"460",
  4 => x"B00", 5 => x"201", 6 => x"40F", 7 => x"A00",
  8 => x"100", 9 => x"B00",
  others => x"000"
);
instr <= ROM(to_integer(unsigned(addr)));
```

6. Control Unit and control truth table

The Control Unit decodes opcode[3:0] and creates the controls needed by the datapath and registers. It should be combinational: give all outputs safe default values first, then override the outputs for recognized opcodes. This avoids unintended latch inference.



Instruction	Opcode	ACC write	R write	Z update	ACC source / function	PC
LDI imm	0001	1	0	1	Immediate	PC+1
ADD imm	0010	1	0	1	ALU: ACC + imm	PC+1
STA	1010	0	1	0	ACC copied to R	PC+1
LDR	1011	1	0	1	R	PC+1
AND imm	0100	1	0	1	ALU: ACC AND imm	PC+1
NOP / unsupported	other	0	0	0	Hold ACC	PC+1

The table is an architectural effect table. Your VHDL may use different internal signal names, but the observable behavior must match it.

Combinational process checklist

- Assign defaults to every output before the case statement.
- Use only the five assigned opcodes.
- STA enables R but disables ACC and Z writes.
- LDR selects R as the ACC input and updates Z from that value.
- Unknown opcodes safely behave as NOP.
- Do not make PC depend on an instruction that is not in Set 4.

7. ALU and datapath

For Set 4, the arithmetic/logic unit needs only ADD and AND. LDI and LDR are data-selection operations rather than ALU calculations. STA writes the current ACC value into R.

ALU select (example)	Operation	8-bit result
00	ADD	ACC + operand; discard carry-out
01	AND	ACC AND operand
10 / 11	Unused / safe default	0 or defined safe value; ACC write must remain disabled for unsupported opcodes

ADD example

ACC=0x1E and operand=0x22. 0x1E + 0x22 = 0x40, so the ALU returns 01000000. The result is written to ACC at the next rising edge and Z becomes 0.

AND example

ACC=0x40 (01000000) and operand=0x60 (01100000). Bitwise AND produces 01000000 = 0x40. Z becomes 0 because the result is nonzero.

Z flag calculation

For operations that write ACC, compute Z_next = 1 when the selected new ACC value equals 0x00, otherwise Z_next = 0. Do not update Z for STA or NOP.

```
case alu_sel is
  when ALU_ADD =>
    alu_result <= std_logic_vector(
      unsigned(acc_in) + unsigned(operand));
  when ALU_AND =>
    alu_result <= acc_in and operand;
  when others =>
    alu_result <= (others => '0');
end case;
```

For ADD, the result is assigned to an 8-bit vector, so overflow beyond bit 7 is discarded. If you use a wider intermediate, explicitly select bits 7 downto 0.

8. ACC, R and Z registers

The ACC, R and Z registers are sequential state. They share CLK and synchronous RESET but have different write enables. ACC is also the externally visible output Q[7:0].

At rising edge	ACC next state	R next state	Z next state
RESET=1	0x00	0x00	1
LDI	operand	hold	operand==0 ? 1 : 0
ADD	ACC+operand (8-bit)	hold	result==0 ? 1 : 0
AND	ACC AND operand	hold	result==0 ? 1 : 0
STA	hold	ACC	hold
LDR	R	hold	R==0 ? 1 : 0
NOP / unsupported	hold	hold	hold

Register enable pattern

Use a rising-edge process. Check RESET first. If RESET is high, clear ACC and R and set Z=1. Otherwise, update each register only when its own enable is asserted. When an enable is not asserted, the register retains its previous value.

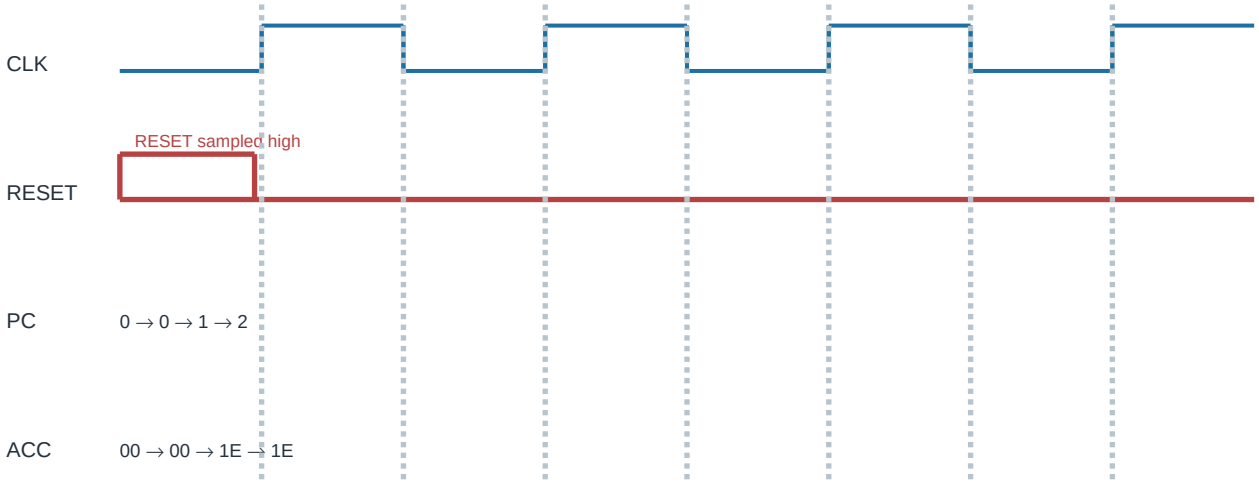
```
process(CLK)
begin
  if rising_edge(CLK) then
    if RESET = '1' then
      ACC <= (others => '0');
      R   <= (others => '0');
      Z   <= '1';
    else
      if acc_we = '1' then
        ACC <= acc_next;
      end if;
      if r_we = '1' then
        R <= ACC;
      end if;
      if z_we = '1' then
        if acc_next = x"00" then Z <= '1';
        else Z <= '0';
        end if;
      end if;
    end if;
  end if;
end process;
```

Common bug to avoid

STA must not update ACC or Z. If you accidentally set z_we for every opcode, the Z flag can change during STA even though the assignment requires it to remain unchanged.

9. Clock, reset and timing diagram

The assignment requires rising-edge state changes and a synchronous active-high reset. A reset value is applied only when RESET is sampled high at a rising edge. Raising RESET between edges does not immediately clear registers.



Illustrative timing only; values shown after active edges. State holds between rising edges.

Signal / event	Required behavior
CLK rising edge	Only point where PC, ACC, R and Z commit next state
RESET=1 at rising edge	PC=0, ACC=0x00, R=0x00, Z=1; ignore instruction execution
RESET=0 at rising edge	Execute instruction at ROM[PC], update enabled registers, PC increments
Between rising edges	Registers hold values; combinational wires may settle
Falling edge	No architectural state update in this project

How to explain this in viva

“The PC selects an instruction from combinational ROM. The control and datapath settle during the cycle. At the rising edge, the enabled registers capture their next values. RESET is synchronous and has priority, so if it is high at the edge the CPU returns to PC=0, ACC=0, R=0 and Z=1.”

Timing caveat

The waveform shown is illustrative rather than simulator-generated. For the report submission, capture a real simulator waveform with CLK, RESET, PC, Q/ACC, Z and R.

10. Truth tables and worked examples

Unlike Approach 1, this path does not derive gates from NAND. Truth tables are still useful to verify the behavior of the ALU functions and register control.

A	B	A AND B	A OR B	A XOR B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

The CPU implements AND from this table; OR and XOR appear only for general logic reference and are not implemented in Set 4.

Register control truth table

RESET on edge	ACC write enable	R write enable	Result
1	X	X	ACC=0, R=0, Z=1
0	1	0	ACC captures acc_next
0	0	1	R captures old ACC; ACC holds
0	0	0	ACC and R hold

Worked data examples

Operation	Input	Result	Z
LDI 0x1E	immediate=00011110	ACC=0x1E	0
ADD 0x22	ACC=0x1E	ACC=0x40	0
AND 0x60	ACC=0x40	ACC=0x40	0
LDI 0x00	immediate=00000000	ACC=0x00	1
LDR	R=0x0F	ACC=0x0F	0
ADD 0x01	ACC=0xFF	ACC=0x00 (carry discarded)	1

Interactive truth-table practice

Use the selectable truth-table widget below to test AND/OR/XOR/XNOR behavior. It is a learning aid; your submitted PDF and VHDL remain the source of truth for Set 4.

11. Exact Set 4 ROM contents

These are the exact ROM words from Appendix B for student set 4. Keep them unchanged in the submitted ROM. The hexadecimal word is opcode concatenated with operand.

Address	Instruction	Opcode	Operand	12-bit word
0	LDI 0x1E	0001	0x1E	0x11E
1	STA	1010	0x00	0xA00
2	ADD 0x22	0010	0x22	0x222
3	AND 0x60	0100	0x60	0x460
4	LDR	1011	0x00	0xB00
5	ADD 0x01	0010	0x01	0x201
6	AND 0x0F	0100	0x0F	0x40F
7	STA	1010	0x00	0xA00
8	LDI 0x00	0001	0x00	0x100
9	LDR	1011	0x00	0xB00
10–15	NOP	0000	0x00	0x000

Encoding reminder

LDI 0x1E = 0001 00011110 = 0x11E. STA = 1010 00000000 = 0xA00. LDR = 1011 00000000 = 0xB00. Use 12-bit constants (for example x"11E") rather than storing only the opcode or only the operand.

12. Cycle-by-cycle expected results

Use the following trace to validate simulation. The state values shown are after each rising edge. PC executed is the address read before the edge; Next PC is the PC state after the edge.

Cycle	PC executed	Instruction	ACC / Q	Z	R	Next PC
0	—	RESET=1	00	1	00	0
1	0	LDI 1E	1E	0	00	1
2	1	STA	1E	0	1E	2
3	2	ADD 22	40	0	1E	3
4	3	AND 60	40	0	1E	4
5	4	LDR	1E	0	1E	5
6	5	ADD 01	1F	0	1E	6
7	6	AND 0F	0F	0	1E	7
8	7	STA	0F	0	0F	8
9	8	LDI 00	00	1	0F	9
10	9	LDR	0F	0	0F	10

Manual trace

1) LDI loads 1E. 2) STA saves 1E in R. 3) ADD gives 1E+22=40. 4) AND 60 leaves 40. 5) LDR restores 1E. 6) ADD 01 gives 1F. 7) AND 0F gives 0F. 8) STA saves 0F. 9) LDI 00 sets ACC=00 and Z=1. 10) LDR restores 0F and Z becomes 0.

Expected final state after instruction at address 9

ACC/Q = 0x0F, R = 0x0F, Z = 0 and PC = 10 (binary 1010).

13. VHDL project structure and implementation order

Keep the source modular so each entity can be compiled and debugged separately.

File	Contents	Unit test / verification
pc.vhd	4-bit PC entity	Reset to 0; increments each rising edge
program_rom.vhd	16 × 12-bit constant ROM	Compare all 10 used words and NOP fill
control_unit.vhd	Opcode decode and enables	Compare each opcode against control table
alu.vhd	ADD and AND combinational logic	Test representative vectors and zero results
registers.vhd	ACC, R and Z state	Reset, enables, hold and Z behavior
cpu_top.vhd	Top-level wiring	Check widths, ports and connected control signals
tb_cpu.vhd	Clock/reset/testbench	Run reset plus 10 instruction edges

Recommended implementation order

1. Define opcode constants and vector widths. 2. Implement PC and test reset/increment. 3. Implement ROM and verify words. 4. Implement ALU ADD/AND. 5. Implement ACC/R/Z registers and write enables. 6. Implement control decode. 7. Connect modules in cpu_top. 8. Write the testbench. 9. Compare every post-edge state with Section 12. 10. Capture waveform and prepare the report.

Compile order

Compile package/constants first if used, then PC, ROM, ALU, Control Unit, registers, top-level CPU and finally the testbench. Entity names and port names must match the component instantiations.

14. VHDL module templates and integration patterns

The snippets below illustrate safe VHDL patterns. Adapt port names consistently across your own entities. They are templates, not a substitute for compiling the complete project in your simulator.

PC sequential pattern

```
process(CLK)
begin
  if rising_edge(CLK) then
    if RESET = '1' then
      PC_reg <= (others => '0');
    else
      PC_reg <= std_logic_vector(unsigned(PC_reg) + 1);
    end if;
  end if;
end process;
PC_out <= PC_reg;
```

ROM combinational pattern

```
process(addr)
begin
  case addr is
    when "0000" => instr <= x"11E";
    when "0001" => instr <= x"A00";
    when "0010" => instr <= x"222";
    when "0011" => instr <= x"460";
    when "0100" => instr <= x"B00";
    when "0101" => instr <= x"201";
    when "0110" => instr <= x"40F";
    when "0111" => instr <= x"A00";
    when "1000" => instr <= x"100";
    when "1001" => instr <= x"B00";
    when others => instr <= x"000";
  end case;
end process;
```

Control-unit case pattern

```
-- Set safe defaults first
acc_we <= '0'; r_we <= '0'; z_we <= '0';
alu_sel <= ALU_ADD; acc_src <= SRC_ALU;

case opcode is
  when "0001" => -- LDI
    acc_we <= '1'; z_we <= '1'; acc_src <= SRC_IMM;
  when "0010" => -- ADD
    acc_we <= '1'; z_we <= '1'; acc_src <= SRC_ALU;
    alu_sel <= ALU_ADD;
  when "1010" => -- STA
    r_we <= '1';
  when "1011" => -- LDR
    acc_we <= '1'; z_we <= '1'; acc_src <= SRC_R;
  when "0100" => -- AND
    acc_we <= '1'; z_we <= '1'; acc_src <= SRC_ALU;
    alu_sel <= ALU_AND;
  when others =>
    null; -- NOP: safe defaults remain
end case;
```

Important integration details

• The PC module needs a defined next-PC interface if increment logic is kept outside the PC entity. The simple pattern above increments internally because Set 4 has no branch instruction.
• The ACC input mux must select immediate, ALU result or R according to the instruction.
• R captures the current ACC for STA at the edge; do not feed the already-updated ACC value back into the same edge expectation.
• The Z calculation must use the value being written to ACC, not the old ACC value.
• Use IEEE.STD_LOGIC_1164.ALL and IEEE.NUMERIC_STD.ALL for unsigned arithmetic.

15. Testbench plan and waveform checklist

The testbench is simulation-only; it does not synthesize into CPU hardware. It generates the clock, applies reset for one rising edge, allows one edge per instruction and observes the state.

Testbench sequence

1. Initialize CLK=0 and RESET=0. 2. Generate a periodic clock. 3. Assert RESET=1 across a rising edge. 4. Deassert RESET=0. 5. Run 10 rising edges for ROM addresses 0–9. 6. Inspect outputs after each edge. 7. Compare with the expected trace. 8. Optionally assert final ACC=0x0F, R=0x0F, Z=0, PC=10.

Signal	Expected observation
CLK	Regular clock; rising edges are state-update points
RESET	High across exactly one active rising edge, then low
PC[3:0]	0 after reset, then 1, 2, …, 10 after the program
Q[7:0] / ACC	00 → 1E → 1E → 40 → 40 → 1E → 1F → 0F → 0F → 00 → 0F
Z	1 after reset; 0 after LDI 1E; 1 after LDI 00; 0 after LDR 0F
R[7:0]	00 → 1E at first STA; remains 1E until second STA; then 0F

VHDL testbench assertions

```
-- Illustrative final-state checks after the final instruction edge
assert Q = x"0F" report "ACC final value mismatch" severity error;
assert R_dbg = x"0F" report "R final value mismatch" severity error;
assert Z = '0' report "Z final value mismatch" severity error;
assert PC = "1010" report "PC final value mismatch" severity error;
```

If R is not a top-level output, expose it through a debug-only port or inspect the internal signal in the simulator. The assignment explicitly requires top-level Q, Z and PC; R should still be verified.

16. Debugging checklist

Symptom	Likely cause	Check
PC stays at 0	RESET remains high or PC process is incomplete	Deassert RESET after one sampled rising edge
ACC updates one cycle late	Incorrect combinational mux or sequential ordering	Compute acc_next before edge; capture at edge
STA changes ACC	ACC enable asserted for STA	STA should set r_we only
STA changes Z	Z enable asserted for STA	z_we=0 for STA
LDR loads wrong value	R not written on STA or source mux wrong	R captures old ACC at STA edge; LDR selects R
Z is wrong	Z based on old ACC instead of new result	Compare acc_next with x"00"
ROM instruction malformed	Wrong word width / opcode bits	Verify 12-bit encodings in Section 11
Simulation has X/U values	Uninitialized state or incomplete assignments	Reset state; assign defaults in combinational processes
Unintended latch inferred	Output omitted on a decode branch	Assign defaults before case
ADD result is too wide	Carry included in ACC	Keep only 8-bit result
PC jumps unexpectedly	Extra branch or bad PC wiring	Set 4 has no JZ; always increment

Golden rule

Debug one module at a time. First validate PC and ROM independently; then ALU; then register enables; then full CPU. Do not try to debug the whole CPU only from the final ACC value.

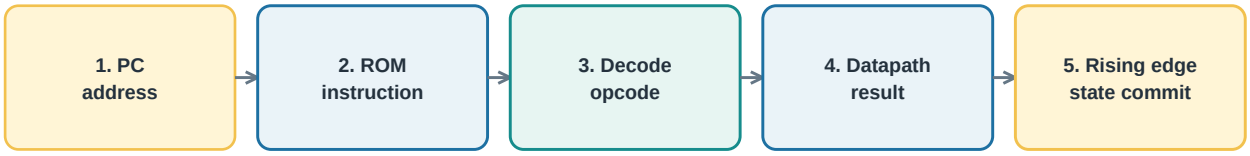
17. Viva questions and concise answers

Question	Answer
Why modular VHDL?	Each module has one responsibility, so it is easier to test, reuse and debug.
Is NAND-only construction required?	No. The uploaded assignment requires modular VHDL entities; NAND-only gate construction is not specified as a requirement.
What is a single-cycle CPU?	Each instruction completes its fetch/decode/execute dataflow within one clock cycle, with state committed at the rising edge.
Why is PC 4 bits?	Four bits represent 16 ROM addresses, 0 through 15.
Why is ROM combinational?	The brief says it is read using PC and does not need a clock.
What does LDI do?	Loads the immediate operand into ACC and updates Z.
What does ADD do?	Adds operand to ACC, stores the low 8 bits and updates Z.
What does STA do?	Copies ACC into R; ACC and Z remain unchanged.
What does LDR do?	Copies R into ACC and updates Z from the loaded value.
What does AND do?	Computes ACC AND immediate operand, writes ACC and updates Z.
When is Z set?	Z=1 when the newly written ACC value is 0x00; reset also sets Z=1.
What is synchronous reset?	Reset is acted on only at the active clock edge when RESET is high.
Why does reset have priority?	The assignment requires reset to override instruction execution.
What happens to carry in ADD?	Carry beyond bit 7 is discarded; arithmetic wraps modulo 256.
What does NOP do?	No ACC/R/Z data change; PC advances by one.
What should the final Set 4 state be?	ACC=0x0F, R=0x0F, Z=0, PC=10.

Presentation tip

Answer in three parts: define the concept, state how this CPU uses it, and give one Set 4 example. For example: “STA is a data-movement instruction. It copies ACC to R at the rising edge. ACC and Z hold their values, which is why the trace shows ACC=1E and Z=0 after STA at address 1.”

18. Final presentation pipeline and conclusion



Single-cycle design: stages are conceptual parts of one instruction cycle, not separate pipeline registers.

Step	What to explain	Evidence to show
1. Specification	Widths, Set 4 instruction subset	Assignment requirements table
2. PC + ROM	Address selects one 12-bit instruction	ROM content table
3. Decode	Opcode creates enables/selects	Control truth table
4. Datapath	LDI, ADD, AND, STA, LDR data routes	Datapath diagram and examples
5. Sequential state	Rising edge, reset priority, register holds	Clock/reset diagram
6. Verification	All 10 instruction cycles match expected trace	Simulator waveform + assertions

Conclusion

The modular VHDL approach satisfies the assignment by implementing the specified CPU blocks as separate entities and connecting them in a top-level design. Correctness depends on exact Set 4 ROM contents, correct decode and data selection, synchronous reset priority, rising-edge state updates, and the specified ACC/R/Z behavior. The final verification should demonstrate that all 10 supplied instructions execute with the expected post-edge values.

Submission checklist

- ■ Student ID and Set 4 identified
- ■ Five instructions only: LDI, ADD, STA, LDR, AND
- ■ PC, ROM, Control Unit, ALU, ACC/R/Z, top-level CPU and testbench included
- ■ ROM matches Appendix B exactly
- ■ RESET is synchronous and active-high
- ■ STA preserves ACC and Z
- ■ LDR updates ACC and Z
- ■ Waveform includes CLK, RESET, PC, Q/ACC, Z and R/debug signal
- ■ Every cycle matches the expected trace
- ■ Challenges and conclusion included

Optional interactive practice

The truth-table control below lets you change A, B and the Boolean operation to inspect the output. It is a companion learning aid; it does not alter the CPU specification.